

RNN yoki LSTM yordamida matn yaratish va ikki tomonlama modellar.

Tabiiy tilni qayta ishlash — 9-ma'ruza

A.A. Abdulali

11:30–12:50 · mavzu №9

30-iyun 2026

Prezentatsiya rejasi

- 1 Kirish: RNN bilan matn yaratamiz
- 2 Autoregressiv matn generatsiyasi
- 3 Temperaturali tanlash (temperature sampling)
- 4 Portlovchi gradient(exploding gradient) va gradient clipping
- 5 Ikki tomonlama RNN (Bidirectional RNN, BiRNN)
- 6 Xulosa va amaliyotga ko'prik

Takrorlash: o'tgan darsdan ikki savol

1-savol

L8/P8 da LSTM tasniflashda oxirgi holat h_T dan **bitta sinf** oldik (*ko'p-dan-bir*). Matn **yaratish**da chiqish qayerlarda bo'lishi kerak?

2-savol

L7–L8 da **yo'qoluvchi gradient** (*vanishing gradient*) muammosini ko'rdik. **Portlovchi gradient** (*exploding gradient*) nima va uni qanday jilovlaymiz?

Takrorlash: o'tgan darsdan ikki savol

1-savol

L8/P8 da LSTM tasniflashda oxirgi holat h_T dan **bitta sinf** oldik (*ko'p-dan-bir*). Matn **yaratish**da chiqish qayerlarda bo'lishi kerak?

1-javob

Har vaqt qadamida **keyingi token taqsimoti** chiqadi (*ko'p-dan-ko'p*). Tanlangan token keyingi kirishga qaytariladi — bu **autoregressiv generatsiya**.

2-savol

L7–L8 da **yo'qoluvchi gradient** (*vanishing gradient*) muammosini ko'rdik. **Portlovchi gradient** (*exploding gradient*) nima va uni qanday jilovlaymiz?

2-javob

Gradient normasi juda kattalashadi $\Rightarrow$ yangilanishlar beqarorlashadi. Yechim: **gradient clipping** (*gradientni kesish*) — norma chegaradan oshsa, gradient masshtablanadi.

Siz bugungi dars oxirida quyidagilarni bajara olasiz:

- 1 **Autoregressiv matn generatsiyasi** zanjirini tushuntira olasiz:

bashorat $\rightarrow$ tanlash $\rightarrow$ qaytadan kiritish.

- 2 Sodda logit misolida **temperaturali softmax** (*temperature softmax*) ni qo'lda hisoblay olasiz:

$$T = 1 : p(\text{nlp}) \approx 0.665, \quad T = 0.5 : p(\text{nlp}) \approx 0.867.$$

- 3 **Portlovchi gradient** (*exploding gradient*) nima ekanini tushuntirib, uni **gradient clipping** (*gradientni cheklash*) bilan qanday jilovlashni izohlay olasiz.

- 4 **Ikki tomonlama rekurrent neyron tarmoq** (*Bidirectional RNN, BiRNN*) g'oyasini tushuntirib, uning **chapdan-o'ngga generatsiyadagi** chegarasini ayta olasiz.

L6 dan: softmax. **L7–L8 dan:** RNN/LSTM, yo'qoluvchi gradient (*vanishing gradient*).
Matematikadan: exp, norma $\| \cdot \|$, vektorlarni birlashtirish.

Matem-

Bugungi dars rejasi

Bosqich	Mavzu	Kutilgan natija
1	Autoregressiv generatsiya	Keyingi tokenni bashorat qilish va qayta kiritish
2	Temperaturali tanlash	$p(\text{nlp})$: 0.665 $\rightarrow$ 0.867 misolini tushunish
3	Gradient clipping	Gradient normasi 5 bo'lsa, uni 2 chegarasigacha cheklash
4	Ikki tomonlama RNN (BiRNN)	Chapdan-o'ngga va o'ngdan-chapga kontekstni birlashtirish
–	Xulosa va 9-amaliyotga ko'prik	Amaliyotda quriladigan modelni oldindan ko'rish

Darsning umumiy yo'nalishi

L7–L8 da RNN/LSTM ni asosan **tushunish va tasniflash** uchun ishlatdik. Bugun shu modelni **matn yaratish**ga moslaymiz: keyingi tokenni bashorat qilamiz, tanlaymiz va modelga qayta kiritamiz. Temperatura misolidagi $T = 1$ uchun $p(\text{nlp}) \approx 0.665$ qiymati 9-amaliyotda kod orqali tekshiriladi.

Bugungi darsdagi uch muammo: tanlash, barqarorlik, kontekst

Model qayerda qiynaladi?

1. Ochko'z tanlash takrorlanishga olib keladi

Har qadamda faqat eng ehtimoliy tokenni olsak (*greedy decoding*, *argmax*), model bir xil naqshni takrorlashi mumkin: “men men men ...”

2. Gradient juda kattalashishi mumkin

Uzun ketma-ketliklarda gradient normasi oshib ketadi. Natijada o'qitish beqarorlashadi.

3. Oddiy RNN faqat o'tmishni ko'radi

Chapdan-o'ngga RNN oldingi tokenlarni ko'radi, lekin tokenni teglash kabi vazifalarda o'ng kontekst ham kerak.

Bugungi yechimlar

- ① **Tasodifiy tanlash** (*sampling*) va **temperatura** (*temperature*) orqali ijodiylik darajasini boshqaramiz.
- ② **Gradient clipping** (*gradientni cheklash*) orqali portlovchi gradientni jilovlaymiz.
- ③ **Ikki tomonlama RNN** (*Bidirectional RNN, BiRNN*) orqali chap va o'ng kontekstni birlashtiramiz.

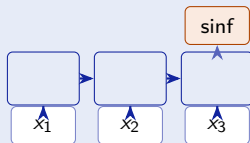
Bugun RNN/LSTM asosida matn yaratish zanjirini, tanlash strategiyasini, o'qitish barqarorligini va ikki tomonlama kontekst g'oyasini bog'laymiz.

Prezentatsiya rejasi

- 1 Kirish: RNN bilan matn yaratamiz
- 2 Autoregressiv matn generatsiyasi
- 3 Temperaturali tanlash (temperature sampling)
- 4 Portlovchi gradient(exploding gradient) va gradient clipping
- 5 Ikki tomonlama RNN (Bidirectional RNN, BiRNN)
- 6 Xulosa va amaliyotga ko'prik

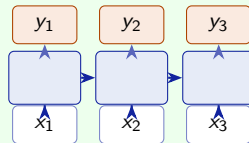
Chiqish qayerda? Ko'p-dan-bir va ko'p-dan-ko'p

Butun ketma-ketlik o'qiladi, ammo **faqat oxirgi yashirin holatdan bitta chiqish** olinadi, ya'ni sinf bashorat qilinadi.



Ko'p-dan-ko'p — generatsiya (bugun)

Har qadamda chiqish bor: har yashirin holatdan **keyingi token taqsimoti** olinadi.



L7–L8 dan ko'prik

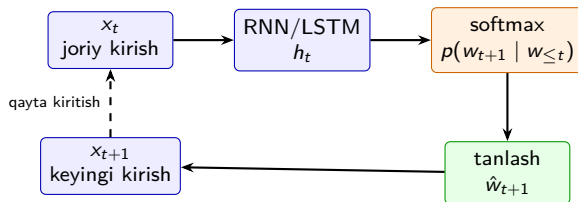
L7–L8 da *many-to-one* arxitekturasini tasniflash uchun, *many-to-many* arxitekturasini esa har qadamda chiqish olish uchun ko'rgan edik. Matn generatsiyasi ham **many-to-many**, lekin bu yerda yana bir muhim farq bor: **tanlangan chiqish keyingi qadam kirishiga aylanadi**. Shu xossa **autoregressiya** deyiladi va keyingi slaydda ko'ramiz.

Intuitsiya: token tanla, uni qayta kirit, davom et

Matn generatsiyasi — zanjirli bashorat

- 1 Joriy kirishdan **keyingi token taqsimoti** olinadi.
- 2 Bitta token **tanlanadi**: argmax yoki tasodifiy tanlash (*sampling*).
- 3 Tanlangan token **keyingi kirish** sifatida qayta beriladi.
- 4 Jarayon tugash belgisi (*EOS, End of Sequence*) kelguncha takrorlanadi.

Autoregressiya: model har qadamda o'zi tanlangan tokenni keyingi qadam uchun kirishga aylantiradi.

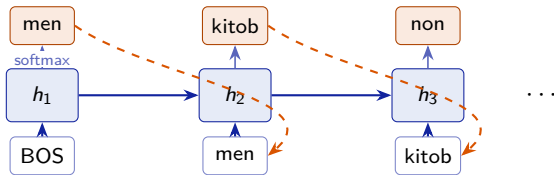


Oddiy fikr: model avval keyingi token ehtimollarini beradi, keyin tanlangan token bilan gapni davom ettiradi.

Autoregressiv generatsiya: rasmiy ta'rif

$$P(w_{1:n}) = \prod_{t=1}^n P(w_t \mid w_{<t}), \quad p_t = \text{softmax}(W_o h_t)$$

Bu yerda: h_t — RNN/LSTM ning yashirin holati; W_o — chiqish qatlami; p_t — **keyingi token taqsimoti**; tanlangan token keyingi qadam kirishiga aylanadi; boshlash uchun maxsus **BOS** belgisi beriladi.



Har qadamda jarayon bir xil: kirish $\rightarrow h_t \rightarrow p_t = \text{softmax}(W_o h_t) \rightarrow$ tanlash $\rightarrow$ qayta kiritish. Shu sababli generatsiya **autoregressiv** deyiladi.

O'tish mantig'i: tasniflashdan generatsiyaga

L7–L8 da butun ketma-ketlik o'qildi va **oxirgi holat** h_T dan bitta chiqish olindi: $h_T \rightarrow \text{sinf}$.
Bu **ko'p-dan-bir** sxema.

2-qadam: generatsiya

Matn yaratishda **har qadamda** chiqish kerak. Har yashirin holatdan keyingi token taqsimoti olinadi: $p_t = \text{softmax}(W_o h_t)$.
Bu **ko'p-dan-ko'p** sxema.

3-qadam: autoregressiya

Taqsimotdan bitta token tanlanadi: $\hat{w}_t \sim p_t$ yoki $\hat{w}_t = \arg \max p_t$. Tanlangan token keyingi qadamga **kirish** sifatida qayta beriladi.

Xulosa

Asosiy RNN/LSTM mexanizmi o'zgarmaydi. Farq shundaki: tasniflashda faqat h_T dan bitta natija olinadi, generatsiyada esa **har qadamdagi chiqish keyingi qadam kirishiga ulanadi**. Shu zanjir **autoregressiv matn generatsiyasini** hosil qiladi.

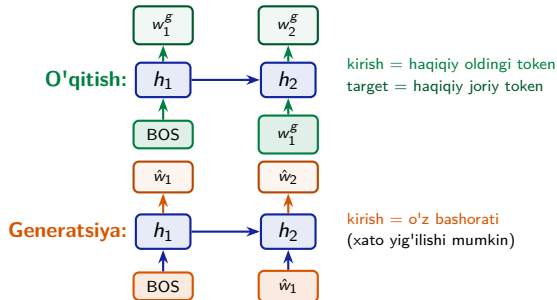
O'qitish va generatsiya: teacher forcing farqi

O'qitish bosqichi

Modelga har qadamda **haqiqiy oldingi token** beriladi:

$$\mathcal{L} = - \sum_{t=1}^T \log P(w_t^{\text{gold}} | w_{<t}^{\text{gold}}).$$

Bu usul *teacher forcing* deb ataladi va **kross-entropiya yo'qotishi** (*cross-entropy loss*) bilan o'qitiladi.



Generatsiya bosqichi

Model o'zi tanlagan tokenni keyingi kirish sifatida ishlatadi:

$$\hat{w}_t \sim P(w_t | \hat{w}_{<t}).$$

Ehtiyot nuqta

O'qitish va generatsiya rejimlari turlicha bo'lgani uchun xatolar qadamdan qadamga yig'ilib borishi mumkin. Bu muammo *exposure bias* deb ataladi.

Asosiy farq: o'qitishda model haqiqiy oldingi token bilan yuradi, generatsiyada esa o'z bashoratiga tayanadi.

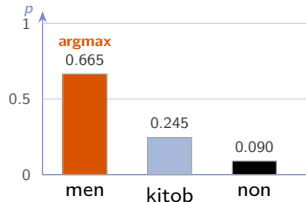
Qo'lda misol: bitta generatsiya qadami

Lug'at = [men, kitob, non]. Modelning softmaxdan oldingi xom ballari: $z = [2.0, 1.0, 0.0]$. Bu soddalashtirilgan o'quv misoli; real modelda lug'at odatda 10^4 – 10^5 token bo'lishi mumkin.

Softmax va greedy tanlov:

$$\begin{aligned} p &= \text{softmax}(z) \\ &= \frac{[e^2, e^1, e^0]}{e^2 + e^1 + e^0} \\ &= \frac{[7.39, 2.72, 1.00]}{11.11} \\ &= [0.665, 0.245, 0.090]. \end{aligned}$$

$$\arg \max p = \text{men}.$$



Greedy tanlov (*greedy decoding*) har doim eng katta ehtimolli tokenni oladi. Shu sababli model bir xil token yoki naqshni takrorlab qolishi mumkin.

Sizning vazifangiz: boshqa logitlar uchun greedy tanlov

Savol

Lug'at = [men, kitob, non], logitlar:

$$z = [0.0, 1.0, 2.0].$$

Ochko'z tanlov (*greedy decoding*, argmax) qaysi tokenni tanlaydi?

Sizning vazifangiz: boshqa logitlar uchun greedy tanlov

Savol

Lug'at = [men, kitob, non], logitlar:

$$z = [0.0, 1.0, 2.0].$$

Ochko'z tanlov (*greedy decoding*, *argmax*) qaysi tokenni tanlaydi?

Javob

Eng katta logit:

$$z_3 = 2.0 \quad \Rightarrow \quad \arg \max(z) = \text{non}.$$

Softmax tartibni o'zgartirmaydi: eng katta logit eng katta ehtimolga aylanadi. Shuning uchun greedy tanlovda softmaxni to'liq hisoblash shart emas.

Kod $\leftrightarrow$ formula: autoregressiv generatsiya

```
import numpy as np

def generate(model, h, start, n):
    seq = [start]
    x = start
    for _ in range(n):
        logits, h = model.step(h, x)      #  $z_t = W_o h_t$ 
        logits = logits - np.max(logits)
        expz = np.exp(logits)
        p = expz / expz.sum()              #  $\text{softmax}(z_t)$ 
        x = int(np.argmax(p))              #  $\text{greedy tanlov}$ 
        seq.append(x)
    return seq
```

Kod $\rightarrow$ formula

Kod	Ma'nosi
model.step	$h_t, z_t = W_o h_t$
expz/sum	$p_t = \text{softmax}(z_t)$
argmax(p)	$\hat{w}_t = \arg \max p_t$
x = ...	$\hat{w}_t$ keyingi kirish

Bu kod bitta autoregressiv qadamni takrorlaydi:

$$x_t \rightarrow h_t \rightarrow z_t \rightarrow p_t \rightarrow \hat{w}_t \rightarrow x_{t+1}.$$

9-amaliyotda TextGenerator shu zanjirni belgi darajasida (*character-level*) quradi.

Generatsiya cheklovi: greedy tanlov takrorlanishga olib keladi

Muammo: takrorlanish halqasi

Ochko'z tanlov (*greedy decoding*) har qadamda eng ehtimolli tokenni tanlaydi:

$$\hat{w}_t = \arg \max_i p_t(i).$$

Agar bir tokenning ehtimoli doim yuqori bo'lsa, model uni qayta-qayta tanlab qolishi mumkin.

Masalan, eng katta ehtimol doim **men** tokenida bo'lsa, greedy tanlov yana **men** ni oladi.

Xulosa: greedy tanlov barqaror, lekin takrorlanishga moyil; sampling esa boshqa variantlarga ham imkon beradi.

Yechim: sampling

Tasodifiy tanlash (*sampling*) tokenni taqsimotdan tanlaydi:

$$\hat{w}_t \sim p_t.$$

Shuning uchun pastroq ehtimolli tokenlar ham ba'zan tanlanadi.

Keyingi bo'limda **temperatura** (*temperature*) sampling qanchalik qat'iy yoki xilma-xil bo'lishini boshqaradi.

Prezentatsiya rejasi

- 1 Kirish: RNN bilan matn yaratamiz
- 2 Autoregressiv matn generatsiyasi
- 3 Temperaturali tanlash (temperature sampling)**
- 4 Portlovchi gradient(exploding gradient) va gradient clipping
- 5 Ikki tomonlama RNN (Bidirectional RNN, BiRNN)
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: temperatura — xilma-xillik regulyatori

Asosiy g'oya

Logitlarni softmaxdan oldin temperatura T ga bo'lamiz: $[p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}, \quad T > 0.]$

T logitlar orasidagi farqni kuchaytiradi yoki kamaytiradi. Shu orqali modelning tanlovi qanchalik qat'iy yoki xilma-xil bo'lishi boshqariladi.

T qanday ta'sir qiladi?

$$T < 1$$

Taqsimot **o'tkirlashadi**: model eng ehtimolli tokenni ko'proq tanlaydi.

$$T = 1$$

Oddiy softmax: logitlar o'zgartirilmaydi.

$$T > 1$$

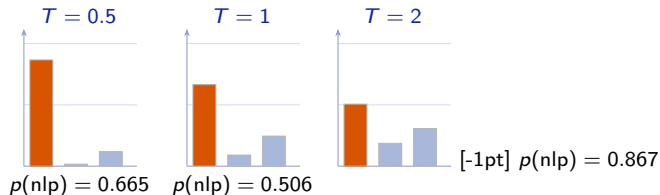
Taqsimot **tekislanadi**: past ehtimolli tokenlar ham ko'proq imkoniyat oladi.

Chegara holatlar: $T \rightarrow 0^+$ — greedy tanlovga yaqinlashadi; $T \rightarrow \infty$ — taqsimot deyarli bir tekis bo'ladi. Demak, temperatura **qat'iylik** va **xilma-xillik** orasidagi murosani boshqaradi.

Temperaturali softmax: rasmiy ta'rif

$$[p_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}, \quad T > 0.]$$

Bu yerda: z_i — i -token logiti; T — temperatura; $T = 1$ — oddiy softmax; $T < 1$ — taqsimotni o'tkirlashtiradi; $T > 1$ — taqsimotni tekislaydi.



Bir xil logitlar: $z = [3, 1, 2]$, tokenlar [nlp, foydali, qiziq]. T kamayganda taqsimot **o'tkirlashadi**; T oshganda taqsimot **tekislanadi**.

Nega logitlarni T ga bo'lish ishlaydi?

Softmax logitlar orasidagi **farq**ga sezgir. Temperatura qo'shilganda bu farq T ga bo'linadi:

$$\frac{z_i}{T} - \frac{z_j}{T} = \frac{z_i - z_j}{T}.$$

$T < 1$: taqsimot o'tkirlashadi

T kichik bo'lsa, logitlar farqi kattalashadi. Eng ehtimolli token yanada ustun bo'ladi.

$T \downarrow \Rightarrow$ qat'iyroq tanlov

$T > 1$: taqsimot tekislanadi

T katta bo'lsa, logitlar farqi kichrayadi. Boshqa tokenlar ham tanlanish imkoniga ega bo'ladi.

$T \uparrow \Rightarrow$ xilma-xilroq tanlov

Rasmiy formula

$$p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}, \quad T > 0.$$

Xulosa: temperatura logit farqlarini boshqaradi: kichik T — qat'iyroq, katta T — xilma-xilroq generatsiya.

Qo'lda hisob: temperatura $p(\text{nlp})$: $0.665 \rightarrow 0.867$

Tokenlar = [nlp, foydali, qiziq], logitlar $z = [3, 1, 2]$.

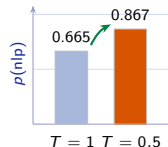
$$e^3 \approx 20.09, \quad e^1 \approx 2.72, \quad e^2 \approx 7.39.$$

$T = 1$ **oddiy softmax**

$$\begin{aligned} p(\text{nlp}) &= \frac{e^3}{e^3 + e^1 + e^2} \\ &= \frac{20.09}{30.20} \approx \mathbf{0.665}. \end{aligned}$$

$T = 0.5$ **ya'ni** $z/T = [6, 2, 4]$

$$\begin{aligned} p(\text{nlp}) &= \frac{e^6}{e^6 + e^2 + e^4} \\ &= \frac{403.4}{465.4} \approx \mathbf{0.867}. \end{aligned}$$



Xulosa: $T = 0.5$ logitlar farqini kattalashtiradi. Shuning uchun eng ehtimolli token **nlp** yanada ustunlashadi:

$$0.665 \rightarrow 0.867.$$

Sizning vazifangiz: $T = 0.5$ da $p(\text{qiziq})$

Savol

Tokenlar = [nlp, foydali, qiziq], logitlar $z = [3, 1, 2]$. $T = 0.5$ bo'lsa:

$$z/T = [6, 2, 4], \quad e^6 \approx 403.4, \quad e^2 \approx 7.39, \quad e^4 \approx 54.60.$$

Yig'indi: $403.4 + 7.39 + 54.60 \approx 465.4$. **$p(\text{qiziq})$ ni toping.**

Sizning vazifangiz: $T = 0.5$ da $p(\text{qiziq})$

Savol

Tokenlar = [nlp, foydali, qiziq], logitlar $z = [3, 1, 2]$. $T = 0.5$ bo'lsa:

$$z/T = [6, 2, 4], \quad e^6 \approx 403.4, \quad e^2 \approx 7.39, \quad e^4 \approx 54.60.$$

Yig'indi: $403.4 + 7.39 + 54.60 \approx 465.4$. $p(\text{qiziq})$ ni toping.

Javob

$$p(\text{qiziq}) = \frac{e^4}{e^6 + e^2 + e^4} = \frac{54.60}{465.4} \approx \mathbf{0.117}.$$

$T = 1$ da $p(\text{qiziq}) \approx 0.245$ edi, $T = 0.5$ da esa 0.117 ga tushdi. Demak, past temperatura yetakchi bo'lmagan tokenlar ehtimolini kamaytiradi.

Kod $\leftrightarrow$ formula: temperaturali softmax

```
import numpy as np

def temp_softmax(z, T=1.0):
    if T <= 0:
        raise ValueError("T > 0 bo'lishi kerak")
    s = np.asarray(z, dtype=float) / T
    s = s - np.max(s)          # barqarorlik uchun
    e = np.exp(s)
    return e / e.sum()

z = [3.0, 1.0, 2.0]
print(temp_softmax(z, 1.0)[0])    # 0.665
print(temp_softmax(z, 0.5)[0])    # 0.867

p = temp_softmax(z, 0.8)
x = np.random.choice(len(p), p=p)
```

Kod $\rightarrow$ formula

Kod	Ma'nosi
z / T	$\tilde{z}_i = z_i / T$
$s - \max(s)$	overflow oldini olish
$\exp(s)$	$e^{\tilde{z}_i - \max(\tilde{z})}$
$e / e.\text{sum}()$	$p_i(T)$
$\text{choice}(\dots)$	$\hat{w} \sim p$

Barqaror softmax: $\max(\tilde{z})$ ni ayirish natijani o'zgartirmaydi, faqat juda katta sonlardan keladigan overflow xavfini kamaytiradi.

Zanjir:

$z \rightarrow z/T \rightarrow \text{softmax} \rightarrow p \rightarrow \text{tanlash.}$

Temperatura cheklovi: chekka qiymatlar xavfli

Chekka holatlar

- **Juda past T :** taqsimot haddan tashqari o'tkirlashadi. Natija greedy tanlovga yaqin: takroriy va zerikarli matn.
- **Juda yuqori T :** taqsimot haddan tashqari tekislanadi. Natija tasodifiy va ma'nosiz tokenlar bo'lishi mumkin.
- **$T = 0$ ishlatilmaydi:** formulada z_i/T bor, shuning uchun kodda $T > 0$ sharti tekshiriladi.

Amaliy tanlov

Amalda $T \in [0.7, 1.0]$ ko'pincha yaxshi boshlang'ich diapazon bo'ladi.

Bu qat'iy qoida emas: mos qiymat model, dataset va vazifaga bog'liq.

Samplingni yanada nazorat qilish uchun: [top-k yoki top-p ; (*nucleus sampling*)] usullari ham ishlatiladi.

Xulosa: juda kichik T matnni takroriy qiladi, juda katta T esa matnni tasodifiylashtiradi. Temperatura o'rtacha qiymatda tanlanadi.

Prezentatsiya rejasi

- 1 Kirish: RNN bilan matn yaratamiz
- 2 Autoregressiv matn generatsiyasi
- 3 Temperaturali tanlash (temperature sampling)
- 4 Portlovchi gradient(exploding gradient) va gradient clipping
- 5 Ikki tomonlama RNN (Bidirectional RNN, BiRNN)
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: gradient ham portlashi mumkin

Muammo: gradient portlashi

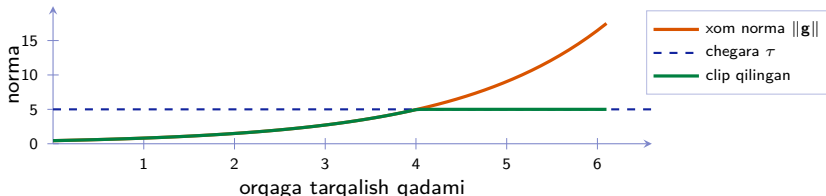
L7–L8 da gradient **so'nishini** (*vanishing gradient*) ko'rgan edik. Teskari muammo ham bor: orqaga tarqalishda gradient ko'p marta **Jakobian matritsalar** bilan ko'paytiriladi. Agar bu ko'paytmalar normasi ko'pincha 1 dan katta bo'lsa, gradient normasi tez o'sishi mumkin: $\|g\| \uparrow\uparrow$. Natijada parametrlar juda katta sakrab yangilanadi, yo'qotish qiymati beqarorlashadi, hatto NaN ham chiqishi mumkin.

Yechim: gradient clipping

Gradient normasi chegara τ dan oshsa, uni qayta masshtablaymiz:

$$g_{\text{clip}} = \min\left(1, \frac{\tau}{\|g\|}\right) g$$

Agar $\|g\| \leq \tau$ bo'lsa, gradient o'zgarmaydi. Aks holda, **yo'nalish saqlanadi**, lekin **kattalik** τ ga tushiriladi.



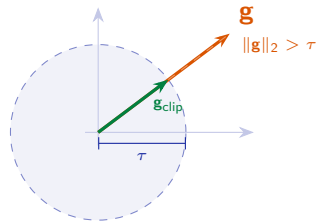
Gradient clipping: rasmiy ta'rif

$$\mathbf{g}_{\text{clip}} = \begin{cases} \mathbf{g}, & \|\mathbf{g}\|_2 \leq \tau, \\ \frac{\tau}{\|\mathbf{g}\|_2} \mathbf{g}, & \|\mathbf{g}\|_2 > \tau. \end{cases}$$

Bu yerda: $\mathbf{g}$ — gradient vektori, $\|\mathbf{g}\|_2$ — uning uzunligi, $\tau > 0$ — ruxsat etilgan maksimal norma.

Agar $\|\mathbf{g}\|_2 > \tau$ bo'lsa, gradient **qayta masshtablanadi**.
Yo'nalish saqlanadi, lekin uzunlik τ dan oshmaydi.

Bu portlovchi gradient ta'sirini kamaytiradi va o'qitishni barqarorroq qiladi.



Yo'nalish bir xil, uzunlik τ gacha cheklanadi.

Xulosa: gradient clipping gradientni nol qilmaydi; faqat juda katta gradientlarni xavfsiz chegaraga keltiradi.

Keltirib chiqarish: clipping nega yo'nalishni saqlaydi?

Agar $\mathbf{g} \neq \mathbf{0}$ bo'lsa:

$$\mathbf{g} = \|\mathbf{g}\|_2 \hat{\mathbf{g}}, \quad \hat{\mathbf{g}} = \frac{\mathbf{g}}{\|\mathbf{g}\|_2}.$$

Bu yerda $\hat{\mathbf{g}}$ — gradient yo'nalishini bildiruvchi birlik vektor.

2-qadam: faqat uzunlik cheklanadi

Agar $\|\mathbf{g}\|_2 > \tau$ bo'lsa:

$$\mathbf{g}_{\text{clip}} = \frac{\tau}{\|\mathbf{g}\|_2} \mathbf{g}.$$

Demak, vektor τ chegarasigacha kichraytiriladi.

Nega yo'nalish saqlanadi?

$$\frac{\tau}{\|\mathbf{g}\|_2} \mathbf{g} = \frac{\tau}{\|\mathbf{g}\|_2} (\|\mathbf{g}\|_2 \hat{\mathbf{g}}) = \tau \hat{\mathbf{g}}.$$

Natijada yo'nalish $\hat{\mathbf{g}}$ o'zgarmaydi, faqat uzunlik τ bo'ladi.

Xulosa: gradient clipping gradientni nol qilmaydi. U faqat juda katta gradientni qayta masshtablaydi: yo'nalish saqlanadi, uzunlik esa τ dan oshmaydi.

Qo'lda misol: gradientni clip qilamiz

$$\|\mathbf{g}\|_2 = \sqrt{3^2 + 4^2} = \sqrt{25} = 5 > 2.$$

Demak, gradient clipping kerak.

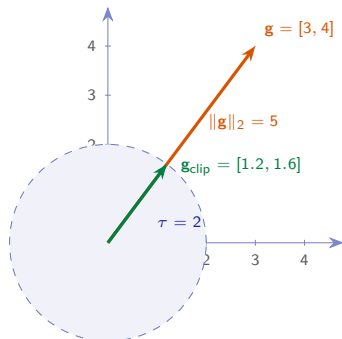
Qayta masshtablash:

$$\mathbf{g}_{\text{clip}} = \frac{\tau}{\|\mathbf{g}\|_2} \mathbf{g} = \frac{2}{5} [3, 4] = [1.2, 1.6].$$

Tekshiruv:

$$\|[1.2, 1.6]\|_2 = \sqrt{1.2^2 + 1.6^2} = \sqrt{1.44 + 2.56} = \sqrt{4} = 2.$$

Xulosa: vektor shu yo'nalishda qoldi, lekin uzunligi 5 dan 2 ga tushdi.



Bir xil yo'nalish, kichikroq uzunlik: $\mathbf{g}$ doira tashqarisidan $\tau = 2$ chegarasigacha qisqaradi.

Sizning vazifangiz: yana bir gradient clipping

Savol

Gradient vektori $\mathbf{g} = [6, 8]$, chegara $\tau = 5$.

Toping:

$$\|\mathbf{g}\|_2 = ? \quad \mathbf{g}_{\text{clip}} = ?$$

Sizning vazifangiz: yana bir gradient clipping

Savol

Gradient vektori $\mathbf{g} = [6, 8]$, chegara $\tau = 5$.

Toping:

$$\|\mathbf{g}\|_2 = ? \quad \mathbf{g}_{\text{clip}} = ?$$

Javob

$$\|\mathbf{g}\|_2 = \sqrt{6^2 + 8^2} = \sqrt{36 + 64} = 10.$$

$$10 > 5 \quad \Rightarrow \quad \mathbf{g}_{\text{clip}} = \frac{\tau}{\|\mathbf{g}\|_2} \mathbf{g} = \frac{5}{10} [6, 8] = [3, 4].$$

Tekshiruv:

$$\|[3, 4]\|_2 = \sqrt{3^2 + 4^2} = 5 = \tau.$$

Yo'nalish saqlandi, uzunlik esa 10 dan 5 ga tushdi.

Kod ↔ formula: gradient clipping

```
import numpy as np

def clip_grad(g, tau):
    if tau <= 0:
        raise ValueError("tau > 0 kerak")
    g = np.asarray(g, dtype=float)
    norm = np.linalg.norm(g)      # L2 norma
    if norm > tau:
        g = g * (tau / norm)      # qayta
                                # masshtab
    return g

print(clip_grad([3.0, 4.0], 2))  # [1.2 1.6]

# PyTorch:
# torch.nn.utils.clip_grad_norm_(
#     model.parameters(), tau)
```

Kod → formula

Kod	Ma'nosi
norm(g)	$\ g\ _2$
tau / norm	$\tau / \ g\ _2$
g * (...)	$g_{\text{clip}} = \frac{\tau}{\ g\ _2} g$
return g	clip qilingan gradient

Agar $\|g\|_2 \leq \tau$ bo'lsa, gradient o'zgarmaydi. Agar $\|g\|_2 > \tau$ bo'lsa, faqat uzunlik cheklanadi.

Amalda clipping odatda `loss.backward()` dan keyin, `optimizer.step()` dan oldin bajariladi.

Clipping cheklovi: sababni emas, alomatni cheklaydi

Cheklov

Gradient clipping portlovchi gradientning ta'sirini kamaytiradi, lekin uning asosiy sababini har doim ham yo'qotmaydi.

Masalan, muammo quyidagilardan kelishi mumkin:

- o'rganish tezligi η juda katta;
- yomon initsializatsiya;
- juda uzun ketma-ketlik;
- beqaror arxitektura yoki loss.

To'g'ri yondashuv

Barqaror o'qitish uchun clipping odatda boshqa usullar bilan birga ishlatiladi:

- mos o'rganish tezligi η ;
- yaxshi initsializatsiya;
- LSTM/GRU kabi barqarorroq rekurrent bloklar;
- kerak bo'lsa, qisqartirilgan BPTT.

LSTM/GRU gradient muammolarini yumshatishga yordam beradi, lekin portlovchi gradientni to'liq yo'q qilmaydi. Shuning uchun clipping baribir foydali.

Prezentatsiya rejasi

- 1 Kirish: RNN bilan matn yaratamiz
- 2 Autoregressiv matn generatsiyasi
- 3 Temperaturali tanlash (temperature sampling)
- 4 Portlovchi gradient(exploding gradient) va gradient clipping
- 5 Ikki tomonlama RNN (Bidirectional RNN, BiRNN)
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: ketma-ketlikni ikki yo'nalishda o'qiymiz

Ba'zi tokenning ma'nosi faqat oldingi so'zlardan emas, **keyingi so'zlardan** ham aniqlanadi.

"Olma shahriga bordi."

Bu yerda **Olma** meva emas, joy nomi ekanini o'ngdagi *"shahriga bordi"* konteksti bildiradi.

BiRNN g'oyasi

Ikki tomonlama RNN (*Bidirectional RNN, BiRNN*) bitta ketma-ketlikni ikki marta o'qiydi:

chapdan o'ngga ($\rightarrow$) va ($\leftarrow$) o'ngdanchapga

So'ng har token uchun ikki holat birlashtiriladi.

Natija

Har token uchun ikki tomonlama kontekst olinadi:

(chap kontekst + o'ng kontekst)

Bu ayniqsa quyidagi vazifalarda foydali:

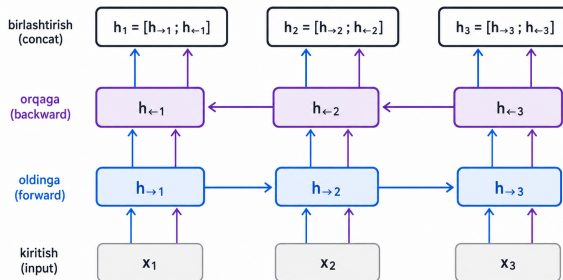
- ketma-ketlik teglash;
- NER — nomli obyektlarni aniqlash (*Named Entity Recognition*);
- matnni tasniflash.

Ehtiyot nuqta: BiRNN butun ketma-ketlik mavjud bo'lganda ishlaydi. Shuning uchun u chapdan-o'ngga real vaqt generatsiyada dekodeer sifatida bevosita ishlatilmaydi.

BiRNN: rasmiy ta'rif

$$\vec{h}_t = \text{RNN}_f(x_t, \vec{h}_{t-1}), \quad \overleftarrow{h}_t = \text{RNN}_b(x_t, \overleftarrow{h}_{t+1}), \quad h_t = [\vec{h}_t; \overleftarrow{h}_t].$$

Bu yerda: $\vec{h}_t$ — oldinga holat, $\overleftarrow{h}_t$ — orqaga holat, $[\ ;]$ — birlashtirish (*concat*).



Har token uchun chap va o'ng kontekst birlashtiriladi.

Eslatma: BiRNN butun ketma-ketlikni talab qiladi. Shuning uchun u autoregressiv generatsiya dekoderi sifatida bevosita ishlatilmaydi.

Keltirib chiqarish: nega ikki holatni birlashtiramiz?

Oldinga RNN t -pozitsiyada faqat oldingi tokenlarni ko'radi:

$$\vec{h}_t \approx x_1, \dots, x_t.$$

Bu **chap kontekst**.

Orqaga RNN t -pozitsiyada keyingi tokenlarni ham hisobga oladi:

$$\overleftarrow{h}_t \approx x_t, \dots, x_T.$$

Bu **o'ng kontekst**.

3-qadam: birlashtirish

Har pozitsiyada ikki holat birlashtiriladi:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t].$$

Token ikki tomonlama kontekst bilan ifodalanadi.

Xulosa

Ikki tomonlama o'qish ketma-ketlik teglash, NER va tasniflashda aniqlikni oshirishga yordam beradi. Lekin BiRNN uchun **butun ketma-ketlik oldindan mavjud bo'lishi kerak**.

Shu sababli BiRNN autoregressiv generatsiya dekoderi sifatida bevosita ishlatilmaydi: generatsiyada kelajak tokenlari hali noma'lum.

Qo'lda misol: ikki tomonlama holatni birlashtiramiz

$$\vec{h}_t = [0.5, 0.2], \quad \overleftarrow{h}_t = [0.1, 0.7].$$

Birlashtirish (concat):

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] = [0.5, 0.2, 0.1, 0.7].$$

$$2 + 2 = 4.$$

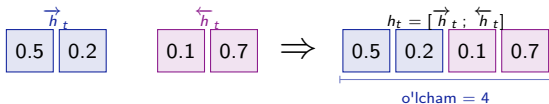
Agar har bir yo'nalish holati d -o'lchamli bo'lsa, birlashtirilgan holat $2d$ -o'lchamli bo'ladi.

Muhim: bu yerda biz vektorlarni **qo'shmaymiz** va **o'rtacha olmaymiz**.

Faqat ularni ketma-ket **ulaymiz**:

$$[0.5, 0.2] ; [0.1, 0.7].$$

Shuning uchun h_t ham chap, ham o'ng kontekstni saqlaydi.



Birlashtirish (concat) — ikki holatni ketma-ket yozishdir.

Sizning vazifangiz: boshqa token uchun birlashtirish

Savol

Bitta token uchun ikki tomonlama holatlar berilgan:

$$\vec{h}_t = [0.9, 0.0], \quad \overleftarrow{h}_t = [0.3, 0.4].$$

Toping: $h_t = [\vec{h}_t; \overleftarrow{h}_t]$ va uning o'lchami.

Sizning vazifangiz: boshqa token uchun birlashtirish

Savol

Bitta token uchun ikki tomonlama holatlar berilgan:

$$\vec{h}_t = [0.9, 0.0], \quad \overleftarrow{h}_t = [0.3, 0.4].$$

Toping: $h_t = [\vec{h}_t; \overleftarrow{h}_t]$ va uning o'lchami.

Javob

Birlashtirishda (*concat*) vektorlar ketma-ket ulanadi:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] = [0.9, 0.0, 0.3, 0.4], \quad \dim(h_t) = 2 + 2 = 4.$$

Ya'ni oldinga holat avval, orqaga holat keyin yoziladi. Bu qo'shish emas, balki **ketma-ket ulash**.

Kod $\leftrightarrow$ formula: bidirectional LSTM

```
import torch.nn as nn

bilstm = nn.LSTM(
    input_size=100,
    hidden_size=64,
    batch_first=True,
    bidirectional=True)

# x: (B, T, 100)
# out: (B, T, 2*64) = (B, T, 128)
out, (h, c) = bilstm(x)
```

Kod $\rightarrow$ formula

Kod	Ma'nosi
bidirectional=True	$\vec{h}_t, \overleftarrow{h}_t$
hidden_size	har yo'n. $H = 64$
out	$[\vec{h}_t; \overleftarrow{h}_t]$
2*64	$2H = 128$

Chiqish: `out.shape = (B, T, 128)`.
Har token uchun oldinga va orqaga holatlar birlashtiriladi.

BiLSTM ketma-ketlik teglash va NER uchun foydali. Autoregressiv generatsiya dekoderi sifatida bevosita ishlatilmaydi, chunki kelajak tokenlari kerak.

Bidirectional cheklovi: autoregressiv generatsiyada bevosita ishlatilmaydi

Sababiyat talabi

Autoregressiv generatsiyada model tokenlarni ketma-ket yaratadi: $[p(w_t | w_{<t})]$

Bu vaqtda $(w_{t+1}, \dots, w_T)$ *halimavjudemas*.

BiRNNning orqaga yo'nalishi esa keyingi tokenlarni ham ko'radi. Shuning uchun u chapdan-o'ngga generatsiya dekoderi sifatida mos emas.

Qoida

BiRNN/BiLSTM butun ketma-ketlik oldindan mavjud bo'lganda foydali:

- ketma-ketlik teglash;
- NER;
- matnni tasniflash.

Autoregressiv dekoder esa sababiyatli va bir tomonlama bo'lishi kerak.

Aniq aytganda: BiRNN encoder sifatida ishlatilishi mumkin, lekin tokenma-token generatsiya qiluvchi dekoder sifatida bevosita ishlatilmaydi.

Prezentatsiya rejasi

- 1 Kirish: RNN bilan matn yaratamiz
- 2 Autoregressiv matn generatsiyasi
- 3 Temperaturali tanlash (temperature sampling)
- 4 Portlovchi gradient(exploding gradient) va gradient clipping
- 5 Ikki tomonlama RNN (Bidirectional RNN, BiRNN)
- 6 Xulosa va amaliyotga ko'prik

O'zbek tili misoli: generatsiya va ikki tomonlama o'qish

1. Badiiy matn generatsiyasi

Agar LSTM badiiy korpusda o'qitilsa, temperatura matn uslubiga ta'sir qiladi:

- ($T < 1$): model tanlovi qat'iyroq bo'ladi; ibora va uslub naqshlari ko'proq takrorlanishi mumkin.
- ($T = 1$): oddiy softmax; model o'rgangan taqsimot bo'yicha token tanlaydi.
- ($T > 1$): tanlov xilma-xilroq bo'ladi; lekin juda katta (T) ma'nosiz birikmalarni oshirishi mumkin.

2. SOV va BiRNN

O'zbek tilida odatiy tartib: [ega ;-; to'ldiruvchi ;-; kesim] ya'ni SOV (*Subject–Object–Verb*).

Masalan: [*“Men kitobni Akmalga berdim.”*]
Kesim ko'pincha gap oxirida keladi. Shuning uchun ayrim tokenlarni to'g'ri tenglashda **keyingi kontekst** ham foydali bo'ladi.

Foyda

BiRNN chap va o'ng kontekstni birlashtiradi. Bu ketma-ketlik tenglash, morfosintaktik tenglash va NER vazifalarida foydali.

Eslatma: temperatura generatsiyada tanlov xilma-xilligini boshqaradi; BiRNN esa asosan mavjud matnni tushunish va tenglashda ishlatiladi.

Sintez: tanlash strategiyasi va yo'nalish

1. Dekodlash: token tanlash

Usul	Xulq
Greedy	qat'iy, lekin takrorlanishga moyil
Past (T)	ishonchliroq, naqshga yopishadi
Yuqori (T)	xilma-xilroq, xato xavfi yuqori

Qachon? Generatsiyada chiqish uslubini boshqarish uchun.

2. Yo'nalish: kontekst qayerdan?

Model	Qachon
Bir tomonlama	autoregressiv generatsiya
BiRNN/BiLSTM	teglash, NER, tasniflash

BiRNN butun ketma-ketlik mavjud bo'lganda chap va o'ng kontekstni birlashtiradi.

Asosiy g'oya

Generatsiya: sababiyatli, chapdan-o'ngga autoregressiv zanjir + greedy/sampling/temperatura orqali tanlash. **Tushunish vazifalari:** BiRNN/BiLSTM orqali chap va o'ng kontekstni birlashtirish. **Barqaror o'qitish:** portlovchi gradient gradient clipping bilan cheklanadi.

Muhim manba: Cho va boshqalar (2014)

Cho, K., van Merriënboer, B., va boshq. (2014). **Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation.** *EMNLP 2014*.

Nega muhim?

- RNN Encoder–Decoder yondashuvini amaliy tarjima vazifasida kuchli ko'rsatdi.
- GRU tipidagi darvozali rekurrent blokni ommalashtirgan muhim ishlardan biri.
- Keyingi attention va Transformer mavzularidagi encoder–decoder fikriga ko'prik bo'ldi.

Bugungi dars bilan bog'lanish

Bu manba ketma-ketlikdan ketma-ketlikka (*sequence-to-sequence*, *seq2seq*) model-lashtirishning muhim asoslaridan biridir.

Bugun esa generatsiyada token tan-lash, temperatura va barqaror o'qitish g'oyalarini ko'rdik.

Muhokama savoli

O'zbek badiiy matnini generatsiya qilishda qaysi temperatura ((0.5,;0.8,;1.2)) tabiiyroq natija beradi deb o'ylaysiz? Nega? P9 da tajribada solishtiramiz.

Bugungi maqsadlar bajarildi

- 1 ✓ **Autoregressiv generatsiya** zanjirini tushuntirdik: bashorat $\rightarrow$ tanlash $\rightarrow$ qaytadan kiritish.
- 2 ✓ **Temperaturali softmax** ni hisobladik:

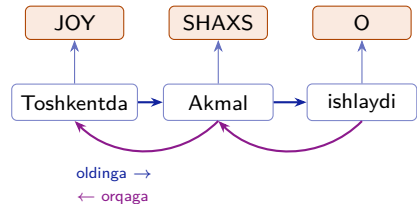
$$T = 1 : p(\text{nlp}) \approx 0.665, \quad T = 0.5 : p(\text{nlp}) \approx 0.867.$$

- 3 ✓ **Portlovchi gradient** va **gradient clipping** g'oyasini tushuntirdik: gradient normasi 5 $\rightarrow$ 2 ga cheklanishini ko'rdik.
- 4 ✓ **Ikki tomonlama RNN** (*Bidirectional RNN, BiRNN*) ni tushuntirdik: $h_t = [\vec{h}_t; \overleftarrow{h}_t]$. Uning autoregressiv generatsiya dekoderni sifatidagi cheklovini ham ko'rdik.

Keyingi ma'ruza (L10): BiRNNning asosiy qo'llanilishi — NER

Bugun ko'rdik: **BiRNN** autoregressiv generatsiya uchun mos emas. Uning **asosiy qo'llanilishi** — *ketma-ketlik teglash* (*sequence labeling*): har token uchun bitta teg bashorat qilinadi.

- **NER** (*Named Entity Recognition*) — matndagi **nomlangan birliklarni** aniqlash: *shaxs, joy, tashkilot* va boshqalar.
- Bu vazifada **butun gap oldindan mavjud** bo'ladi, shuning uchun chap va o'ng kontekstning **ikkalasi** ishlatiladi.

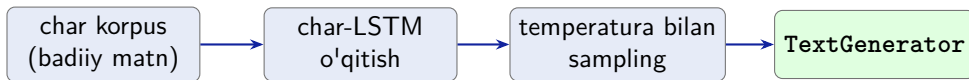


Har token tegini **chap + o'ng kontekst** yordamida oladi.

L10 da **BiLSTM** yordamida har token uchun teg bashorat qilamiz:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \implies \text{token tegi.}$$

Keyingi amaliyot P9: belgi-darajali LSTM bilan generatsiya



P9 da qurasiz:

- TextGenerator: char-LSTM ni `train()` va `generate()` bilan ishlatish
- Turli temperaturada generatsiya:
 $T = 0.3, 0.7, 1.2$
- Temperaturali softmax helper funksiyasini tekshirish

Bu tekshiruv L9 dagi qo'lda hisob bilan mos kelishi kerak:

$$p(\text{nlp}; T = 1) \approx 0.665$$

```
assert abs(p_nlp_T1 - 0.665) < 1e-3
```

Demak, P9 da ikki qism bor: **(1)** temperaturali softmax ni tekshirish, **(2)** shu g'oyani char-LSTM generatsiyasiga ulash.

- ❶ Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y. (2014). *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*. EMNLP 2014.
- ❷ Graves, A. (2013). *Generating Sequences With Recurrent Neural Networks*. arXiv:1308.0850. Matn generatsiyasi va sampling uchun.
- ❸ Pascanu, R., Mikolov, T., Bengio, Y. (2013). *On the difficulty of training recurrent neural networks*. ICML 2013. Portlovchi gradient va gradient clipping uchun.
- ❹ Schuster, M., Paliwal, K. K. (1997). *Bidirectional Recurrent Neural Networks*. IEEE Transactions on Signal Processing. Ikki tomonlama RNN uchun klassik manba.
- ❺ PyTorch hujjatlari: `torch.nn.LSTM(bidirectional=True)` va `torch.nn.utils.clip_grad_norm`.

Ilova S1: top- k va top- p sampling

Temperature — yagona nazorat usuli emas. Samplingni cheklash uchun yana ikki keng tarqalgan usul bor:

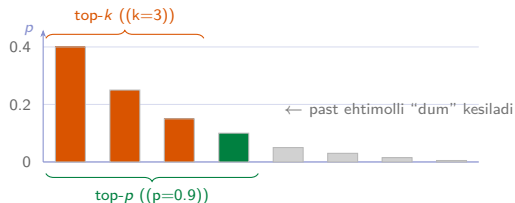
Faqat eng yuqori ehtimolli (k) ta token qoldiriladi.

Qolgan tokenlarning ehtimoli (0) qilinadi, so'ng taqsimot qayta normallanadi.

Top- p / nucleus

Tokenlar ehtimol bo'yicha saralanadi.

Yig'indi ehtimol (p) ga yetguncha eng yuqori tokenlar olinadi. Bu to'plam *nucleus* deyiladi.



Misolda top- k uchta tokenni oladi; top- p esa yig'indi (0.9) bo'lgani uchun to'rtta tokenni oladi.

Ilova S2: temperatura chegaralarini keltirib chiqarish

1. $T \rightarrow 0^+$: maksimum logit ustunlashadi.

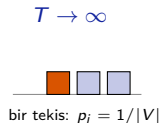
Agar eng katta logit yagona bo'lsa, $z_m = \max_j z_j$, unda:

$$p_m(T) = \frac{e^{z_m/T}}{\sum_j e^{z_j/T}} \rightarrow 1, \quad p_i(T) \rightarrow 0 \quad (i \neq m).$$

Natija: taqsimot one-hot ko'rinishga yaqinlashadi, ya'ni greedy tanlovga o'xshaydi.

2. $T \rightarrow \infty$: barcha logitlar ta'siri yo'qoladi.

$$z_i/T \rightarrow 0 \Rightarrow e^{z_i/T} \rightarrow e^0 = 1 \Rightarrow p_i(T) \rightarrow \frac{1}{|V|}.$$



Xulosa: temperatura $T > 0$ bo'lishi kerak. Kichik T tanlovni qat'iy lashtiradi; juda katta T esa taqsimotni deyarli bir tekis qiladi.